

Calibração dos parâmetros β e γ no modelo SIR: o surto de influenza de 1978

1 Objetivo

Este texto mostra como calibrar os parâmetros β e γ do modelo SIR usando Python. O exemplo é o surto de influenza de 1978 em uma escola interna inglesa. Usaremos os números observados “confined to bed” (B) e “convalescent” (C), tomando

$$I_{\text{obs}}(t_i) = B(t_i) + C(t_i)$$

como uma aproximação observacional para o compartimento $I(t)$.

A calibração é feita por mínimos quadrados, resolvendo a EDO SIR numericamente em cada avaliação dos parâmetros.

2 Modelo SIR

Para uma população fechada de tamanho N ,

$$S' = -\beta SI/N, \tag{1}$$

$$I' = \beta SI/N - \gamma I, \tag{2}$$

$$R' = \gamma I, \tag{3}$$

com $S + I + R = N$. Os parâmetros têm unidade dia^{-1} e

$$R_0 = \frac{\beta}{\gamma}, \quad \text{duração infecciosa média} = \frac{1}{\gamma}.$$

3 Dados

Tomamos $N = 763$ e as seguintes 14 observações:

Data	B	C	$B + C$
22/01	3	0	3
23/01	8	0	8
24/01	26	0	26
25/01	76	0	76
26/01	225	9	234
27/01	298	17	315
28/01	258	105	363
29/01	253	162	415
30/01	189	176	365
31/01	128	166	294
01/02	68	150	218
02/02	29	85	114
03/02	14	47	61
04/02	4	20	24

Assim,

$$I_{\text{obs}} = (3, 8, 26, 76, 234, 315, 363, 415, 365, 294, 218, 114, 61, 24).$$

A soma $B + C$ é uma *proxy* para I : “convalescente” não é necessariamente idêntico ao compartimento infeccioso idealizado do SIR. Por isso, os parâmetros obtidos aqui podem diferir dos parâmetros publicados em outras análises.

4 Condições iniciais

Colocamos $t = 0$ em 22/01 e usamos o primeiro ponto observado:

$$I(0) = 3, \quad S(0) = 760, \quad R(0) = 0.$$

5 Calibração

Para cada (β, γ) resolvemos a EDO e calculamos $I(t_i; \beta, \gamma)$. Minimizamos

$$\Phi(\beta, \gamma) = \sum_i [I(t_i; \beta, \gamma) - I_{\text{obs}}(t_i)]^2, \quad \beta, \gamma \geq 0.$$

O programa usa duas rotinas do SciPy.

5.1 odeint

```
def sir_rhs(y, t, beta, gamma):  
    S, I, R = y  
    dS = -beta*S*I/N  
    dI = beta*S*I/N - gamma*I  
    dR = gamma*I  
    return dS, dI, dR  
  
def infected_curve(t_eval, beta, gamma):  
    sol = odeint(sir_rhs, (S0, I0, R0),  
                  t_eval, args=(beta, gamma))  
    return sol[:, 1]
```

`odeint` integra numericamente o sistema diferencial.

5.2 curve_fit

```
popt, pcov = curve_fit(  
    infected_curve, t, Iobs,  
    p0=(1.5, 0.3),  
    bounds=(0.0, np.inf),  
    maxfev=20000  
)
```

`curve_fit` procura os parâmetros que minimizam a soma dos quadrados dos resíduos. O argumento `p0` é o chute inicial e `bounds` impõe $\beta, \gamma \geq 0$.

6 Resultado

Para as condições e os dados acima, o ajuste dá aproximadamente

$$\hat{\beta} = 1.440772 \text{ dia}^{-1}, \quad \hat{\gamma} = 0.263430 \text{ dia}^{-1},$$

e portanto

$$\hat{R}_0 = 5.469282, \quad 1/\hat{\gamma} = 3.796078 \text{ dias}.$$

A soma dos quadrados dos resíduos é

$$\text{RSS} = 20737.782747.$$

7 Comparação com a Figura 5-2 de White

White apresenta, para sua análise do surto, aproximadamente

$$\beta = 1.97 \text{ dia}^{-1}, \quad \gamma = 0.47 \text{ dia}^{-1}, \quad R_0 = 4.18.$$

Uma nova calibração não precisa reproduzir exatamente esses valores: as condições iniciais, a interpretação de I , o intervalo temporal, a digitalização dos dados e a função objetivo podem ser diferentes.

8 Figura

A figura seguinte mostra os pontos $B + C$ e a curva SIR ajustada.

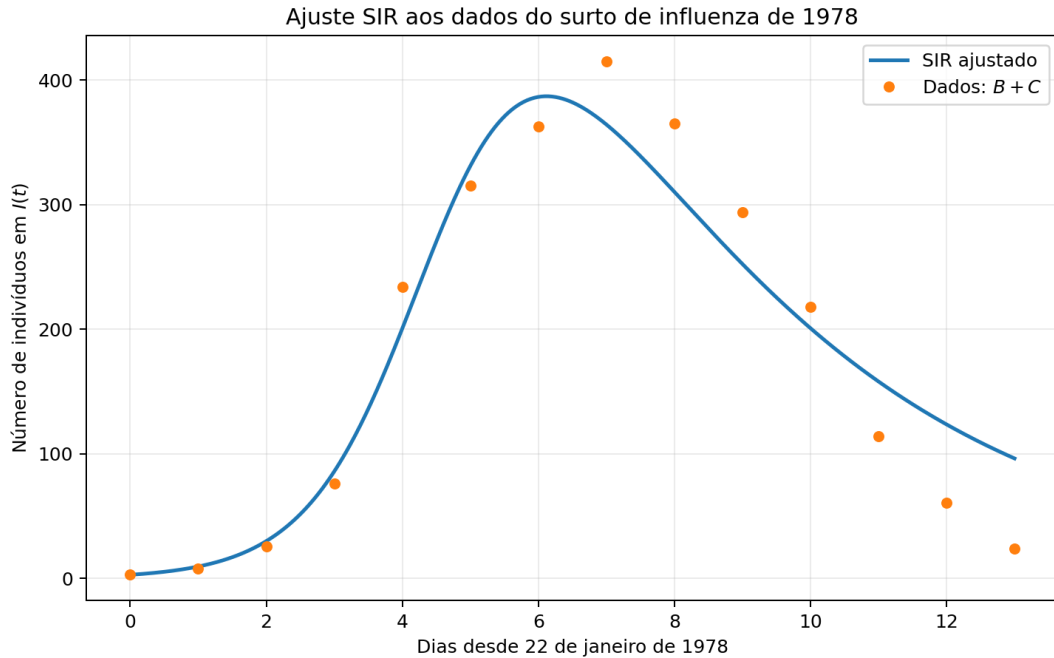


Figura 1: Ajuste SIR aos dados $B + C$ do surto de influenza de 1978, em apresentação análoga à Figura 5-2 de White.

9 Reprodução

Instale NumPy, SciPy e Matplotlib, por exemplo:

```
pip install numpy scipy matplotlib
```

e execute:

```
python calibracao_SIR_surto_BMJ_White.py
```

10 Referências

Referências

- [1] “Influenza in a boarding school”, *British Medical Journal*, 1978.
- [2] P. J. White, “Mathematical Models in Infectious Disease Epidemiology”, in *Infectious Diseases*, Elsevier. A discussão inclui a aplicação do SIR ao surto de influenza de 1978 e a Figura 5-2.
- [3] G. De Vries, T. Hillen, M. Lewis, J. Mueller and B. Schoenfish, *A Course in Mathematical Biology*, SIAM, 2006.
- [4] P. Virtanen et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python”, *Nature Methods* 17 (2020), 261–272, DOI: 10.1038/s41592-019-0686-2.
- [5] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006.